

Homework 05

Question 1: Estimate the bias and variance of KNN

(a)

The true mean response at the target is

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 \approx 0.594218.$$

```
library(FNN)
set.seed(43251)
n = 400
p = 3
k_values = seq(1, 29, 2)
x0 = matrix(c(0.5, 0.7, 1), nrow = 1)
f0 = 0.5*x0[1,1] + sin(x0[1,2]) - 0.3*x0[1,3]^2
X = matrix(rnorm(n*p), nrow = n)
y = 0.5*X[,1] + sin(X[,2]) - 0.3*X[,3]^2 +
  rnorm(n, sd = 0.1)
pred = numeric(length(k_values))
for (m in seq_along(k_values)) {
  knn.fit = knn.reg(X, x0, y, k = k_values[m], algorithm = "brute")
  pred[m] = knn.fit$pred
}
data.frame(k = k_values, prediction = pred, truth = f0)
```

	k	prediction	truth
1	1	0.7888302	0.5942177
2	3	0.6621674	0.5942177
3	5	0.6642853	0.5942177
4	7	0.6100117	0.5942177
5	9	0.6318671	0.5942177
6	11	0.4925114	0.5942177
7	13	0.5222989	0.5942177
8	15	0.5810203	0.5942177
9	17	0.5738171	0.5942177
10	19	0.5751479	0.5942177
11	21	0.5800517	0.5942177
12	23	0.5519177	0.5942177
13	25	0.5214556	0.5942177
14	27	0.4952957	0.5942177
15	29	0.4972667	0.5942177

(b)

Each of the 200 repetitions uses one new training dataset for all values of k . Rows of `pred_matrix` represent repetitions; columns represent k .

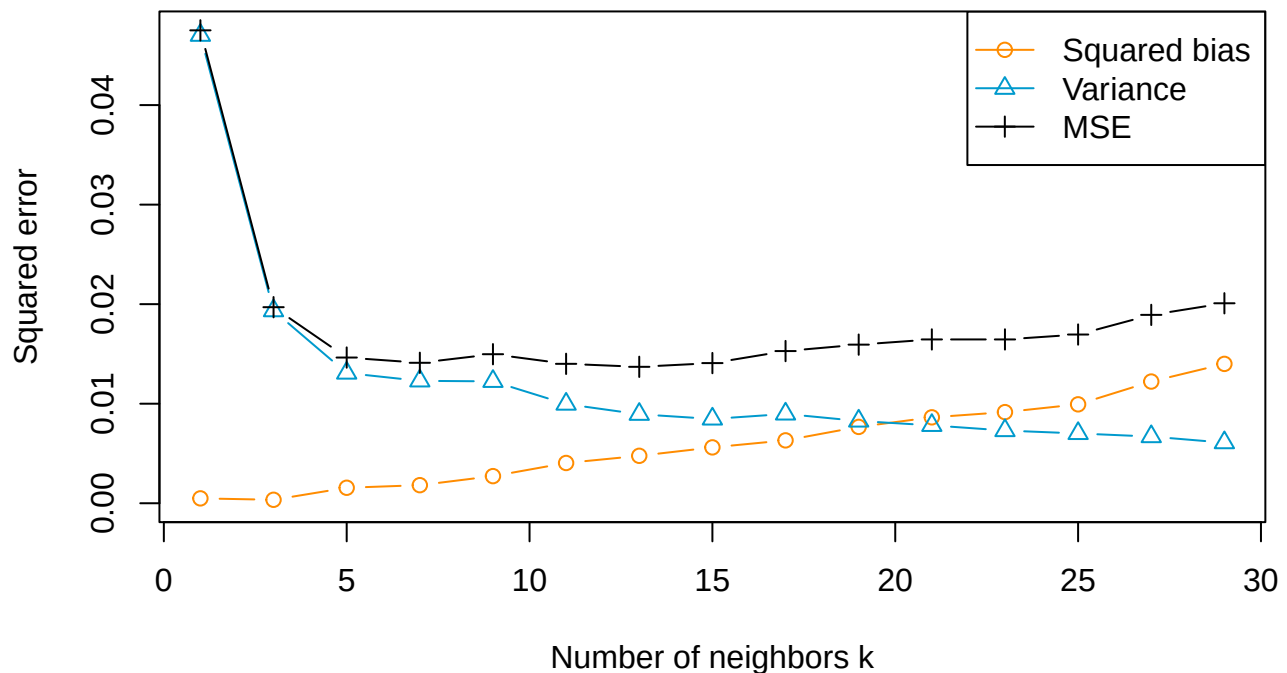
```
set.seed(43252)
nsim = 200
pred_matrix = matrix(NA_real_, nrow = nsim, ncol = length(k_values))
for (l in seq_len(nsim)) {
  X = matrix(rnorm(n*p), nrow = n)
  y = 0.5*X[,1] + sin(X[,2]) - 0.3*X[,3]^2 +
    rnorm(n, sd = 0.1)
  for (m in seq_along(k_values)) {
    knn.fit = knn.reg(X, x0, y, k = k_values[m], algorithm = "brute")
    pred_matrix[l, m] = knn.fit$pred
  }
}
mean_pred = colMeans(pred_matrix)
bias = mean_pred - f0
variance = colMeans(sweep(pred_matrix, 2, mean_pred, "-")^2)
mse = colMeans((pred_matrix - f0)^2)
results = data.frame(k = k_values, bias = bias, squared_bias = bias^2,
                     variance = variance, MSE = mse)
results
```

	k	bias	squared_bias	variance	MSE
1	1	-0.02212635	0.0004895752	0.047019892	0.04750947
2	3	-0.01863344	0.0003472050	0.019348105	0.01969531
3	5	-0.03943681	0.0015552620	0.013082015	0.01463728
4	7	-0.04260658	0.0018153204	0.012295179	0.01411050
5	9	-0.05219684	0.0027245099	0.012247433	0.01497194
6	11	-0.06358861	0.0040435113	0.009961824	0.01400534
7	13	-0.06902699	0.0047647249	0.008940642	0.01370537
8	15	-0.07490990	0.0056114937	0.008468540	0.01408003
9	17	-0.07947875	0.0063168714	0.008971971	0.01528884
10	19	-0.08757292	0.0076690167	0.008264919	0.01593394
11	21	-0.09290960	0.0086321940	0.007825070	0.01645726
12	23	-0.09566891	0.0091525398	0.007304251	0.01645679
13	25	-0.09968339	0.0099367790	0.007012532	0.01694931
14	27	-0.11056815	0.0122253164	0.006698005	0.01892332
15	29	-0.11832666	0.0140011994	0.006087265	0.02008846

```
max_difference = max(abs(mse - (bias^2 + variance)))
max_difference
```

```
[1] 1.040834e-17
```

The variance uses divisor 200. The maximum absolute difference between MSE and squared bias plus variance is $1.04e-17$, verifying the decomposition up to numerical rounding.



As k increases, variance generally decreases because more observations are averaged, while bias tends to increase because more distant observations are included. An intermediate k can therefore minimize MSE. Similarly, stronger regularization in ridge regression and lasso reduces model flexibility and variance but can increase bias.

Question 2: Compare KNN with lasso

(a)

```
library(FNN)
library(glmnet)
set.seed(43252)
n = 400
p = 30
lambda = 10^seq(0, -4, length.out = 41)
results = data.frame(Setting = c("Independent", "Correlated"),
                     KNN_MSE = NA, Lasso_MSE = NA, Lambda = NA, Nonzero = NA)

for (l in 1:2) {
  Sigma = diag(p)
  if (l == 2) Sigma = 0.8 + 0.2*diag(p)
  X = matrix(rnorm((n + 1000)*p), ncol = p) %%% chol(Sigma)
  y = 0.5*X[,1] + sin(X[,2]) - 0.3*X[,3]^2 +
      rnorm(n + 1000, sd = 0.1)
  test.X = X[401:1400,]
  test.y = y[401:1400]
  X = X[1:400,]
  y = y[1:400]
  # KNN
```

```

knn.fit = knn.reg(X, test.X, y, k = 5, algorithm = "brute")
results$KNN_MSE[1] = mean((test.y - knn.fit$pred)^2)
# 10-fold CV for lasso
fold = sample(rep(1:10, each = 40))
cv.mse = matrix(NA, 10, length(lambda))
for (m in 1:10) {
  X.mean = colMeans(X[fold != m,])
  X.fit = sweep(X[fold != m,], 2, X.mean, "-")
  X.sd = sqrt(colMeans(X.fit^2))
  X.fit = sweep(X.fit, 2, X.sd, "/")
  X.valid = sweep(X[fold == m,], 2, X.mean, "-")
  X.valid = sweep(X.valid, 2, X.sd, "/")
  y.mean = mean(y[fold != m])
  lasso.fit = glmnet(X.fit, y[fold != m] - y.mean, alpha = 1,
                    lambda = lambda, intercept = FALSE, standardize = FALSE)
  y.hat = predict(lasso.fit, X.valid, s = lambda) + y.mean
  cv.mse[m,] = colMeans((y[fold == m] - y.hat)^2)
}
lambda.min = lambda[which.min(colMeans(cv.mse))]
# Refit lasso on full training data
X.mean = colMeans(X)
X.fit = sweep(X, 2, X.mean, "-")
X.sd = sqrt(colMeans(X.fit^2))
X.fit = sweep(X.fit, 2, X.sd, "/")
test.X.fit = sweep(test.X, 2, X.mean, "-")
test.X.fit = sweep(test.X.fit, 2, X.sd, "/")
y.mean = mean(y)
lasso.fit = glmnet(X.fit, y - y.mean, alpha = 1, lambda = lambda,
                  intercept = FALSE, standardize = FALSE)
test.pred = as.vector(predict(lasso.fit, test.X.fit, s = lambda.min)) + y.mean
results$Lasso_MSE[1] = mean((test.y - test.pred)^2)
results$Lambda[1] = lambda.min
results$Nonzero[1] =
  sum(coef(lasso.fit, s = lambda.min)[-1,1] != 0)
}
results

```

	Setting	KNN_MSE	Lasso_MSE	Lambda	Nonzero
1	Independent	0.5564800	0.2601745	0.02511886	11
2	Correlated	0.1608514	0.2259450	0.01000000	16

(b)

Lasso has a lower test MSE with independent predictors, whereas KNN has a lower test MSE with correlated predictors. With independent predictors, many irrelevant variables affect the Euclidean distance, which can make KNN choose less useful neighbors. With correlated predictors, the variables share more information, so the neighborhoods may better reflect the relevant predictors. Lasso is different because it can shrink some coefficients to zero instead of using every predictor equally.

(c)

The true model contains nonlinear terms, $\sin(X_2)$ and X_3^2 , while the lasso model uses only linear terms. Therefore, lasso cannot represent the true response function exactly. KNN does not assume a linear form, but its performance depends on the quality of the neighbors. A single dataset in each setting is not enough to conclude that correlation always helps KNN or that one method is always better.

Question 3: Observed dimension and latent dimension

(a)

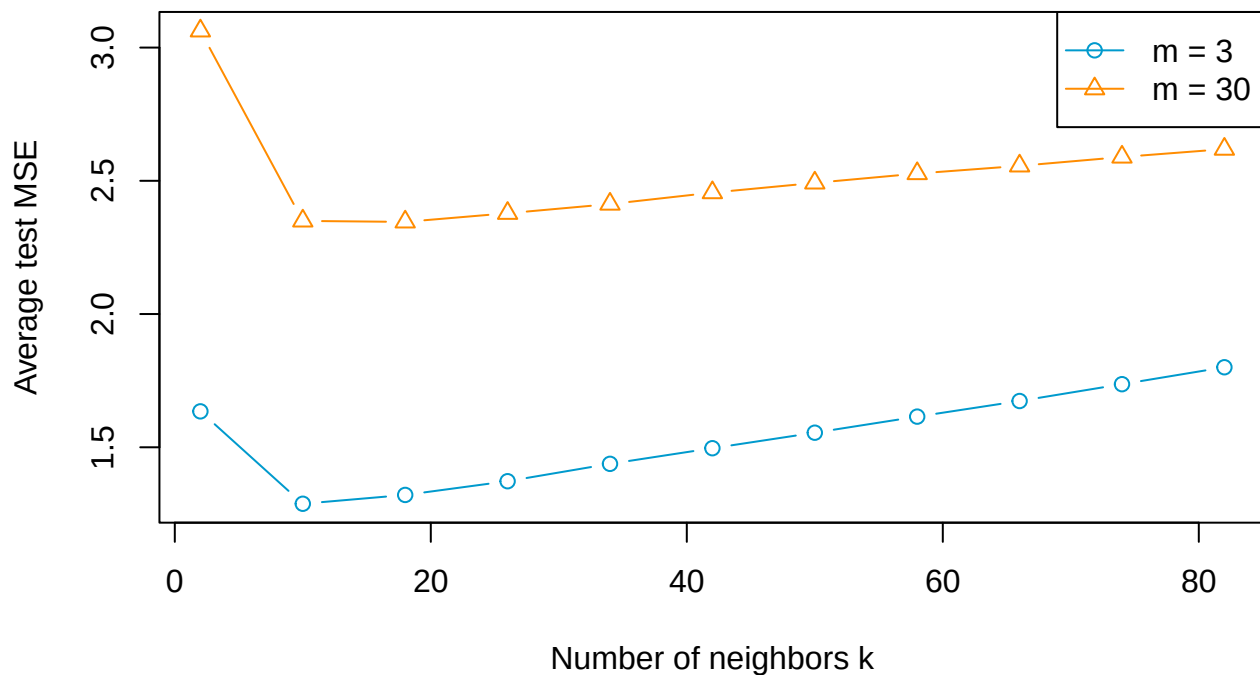
```
library(FNN)
set.seed(43254)
p = 100
m_values = c(3, 30)
k_values = seq(2, 82, 8)
test_mse = matrix(NA, length(k_values), 2)
for (m in m_values) {
  A = matrix(runif(p*m, -2, 2), p, m)
  Z = matrix(rnorm(400*m), 400, m)
  X = Z %*% t(A) + matrix(rnorm(400*p), 400, p)
  y = Z[,1] + Z[,2] + rnorm(400)
  for (k in k_values) {
    knn.fit = knn.reg(X[1:200,], X[201:400,], y[1:200],
                      k = k, algorithm = "brute")
    test_mse[match(k, k_values), match(m, m_values)] =
      mean((y[201:400] - knn.fit$pred)^2)
  }
}
data.frame(k = k_values, m3 = test_mse[,1], m30 = test_mse[,2])
```

	k	m3	m30
1	2	1.449265	3.457456
2	10	1.253345	2.583372
3	18	1.292923	2.533071
4	26	1.379955	2.524038
5	34	1.448416	2.512187
6	42	1.505892	2.560783
7	50	1.590057	2.562911
8	58	1.660186	2.593970
9	66	1.739597	2.607162
10	74	1.816963	2.651181
11	82	1.871802	2.672724

(b)

```
nsim = 50
test_mse = array(NA, c(nsim, length(k_values), 2))
for (l in 1:nsim) {
  for (m in m_values) {
    A = matrix(runif(p*m, -2, 2), p, m)
    Z = matrix(rnorm(400*m), 400, m)
    X = Z %*% t(A) + matrix(rnorm(400*p), 400, p)
    y = Z[,1] + Z[,2] + rnorm(400)
    for (k in k_values) {
      knn.fit = knn.reg(X[1:200,], X[201:400,], y[1:200],
                        k = k, algorithm = "brute")
      test_mse[l, match(k, k_values), match(m, m_values)] =
        mean((y[201:400] - knn.fit$pred)^2)
    }
  }
}
```

```
}
mean_mse = apply(test_mse, c(2,3), mean)
```



(c)

When $m = 3$, most of the structured variation in the 100 observed predictors is driven by only three latent variables, so distances can better reflect the relevant latent structure. When $m = 30$, many additional latent directions affect distances without affecting the response, making KNN neighborhoods less informative.

The noise η_i also changes the observed distances. Without this noise, $\mathbf{AZ}_i$ lies in the column space of $\mathbf{A}$, whose dimension is at most m . Independent noise in all 100 coordinates means that the observed data are not confined to that space.

Question 4: Classify handwritten digits

(a)

```
library(ElemStatLearn)
euclidean_distance = function(x0, X) {
  sqrt(rowSums(sweep(X, 2, x0, "-")^2))
}
manhattan_distance = function(x0, X) {
  rowSums(abs(sweep(X, 2, x0, "-")))
}
X = zip.train[1:1000, -1]
```

```

y = zip.train[1:1000, 1]
X_test = zip.test[1:500, -1]
y_test = zip.test[1:500, 1]
knn_classify = function(X, y, X_test, distance) {
  y_hat = integer(nrow(X_test))
  for (i in 1:nrow(X_test)) {
    d = distance(X_test[i,], X)
    neighbors = order(d, 1:nrow(X))[1:5]
    y_hat[i] =
      which.max(tabulate(y[neighbors] + 1, nbins = 10)) - 1
  }
  y_hat
}

```

(b)

Euclidean test error: 0.122

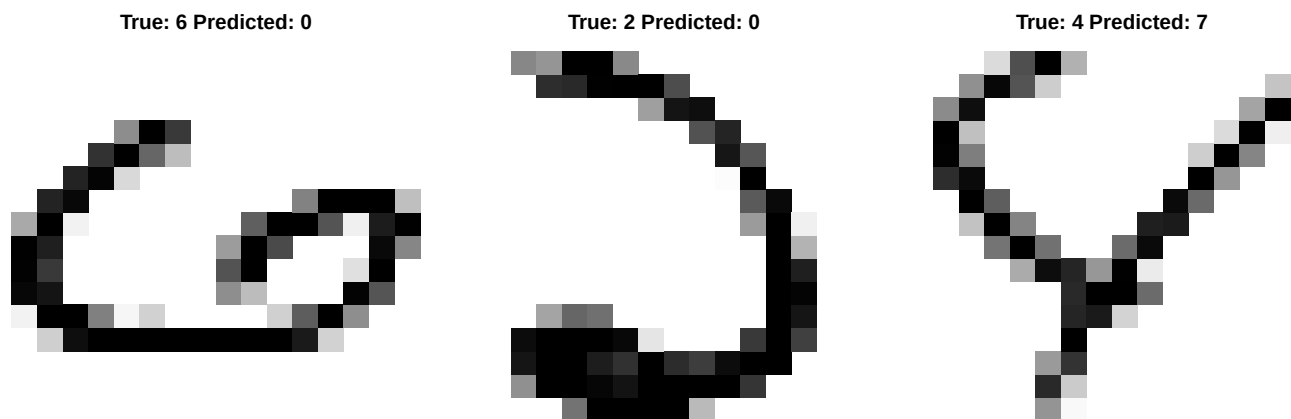
	True									
Predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

Manhattan test error: 0.138

	True									
Predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

Euclidean distance gives a test error of 0.122, while Manhattan distance gives a test error of 0.138. The most common confusion is a true digit 2 being predicted as 0. Other frequent errors include 5 predicted as 0 or 3, 9 predicted as 7, and 4 predicted as 9.

(c)



[1] 22

```
Manhattan_correct
Euclidean_correct FALSE TRUE
FALSE      58      3
TRUE       11     428
```

The first three Euclidean errors are 6 predicted as 0, 2 predicted as 0, and 4 predicted as 7. These mistakes are visually understandable because the shapes are somewhat unusual and resemble the predicted digits.

The two distance measures give different predictions on 22 images. Euclidean is correct while Manhattan is wrong on 11 images, while the reverse occurs on 3 images. On the remaining 8 images, both methods are wrong but predict different digits. Therefore, two methods can have similar overall error rates while still making mistakes on different observations.

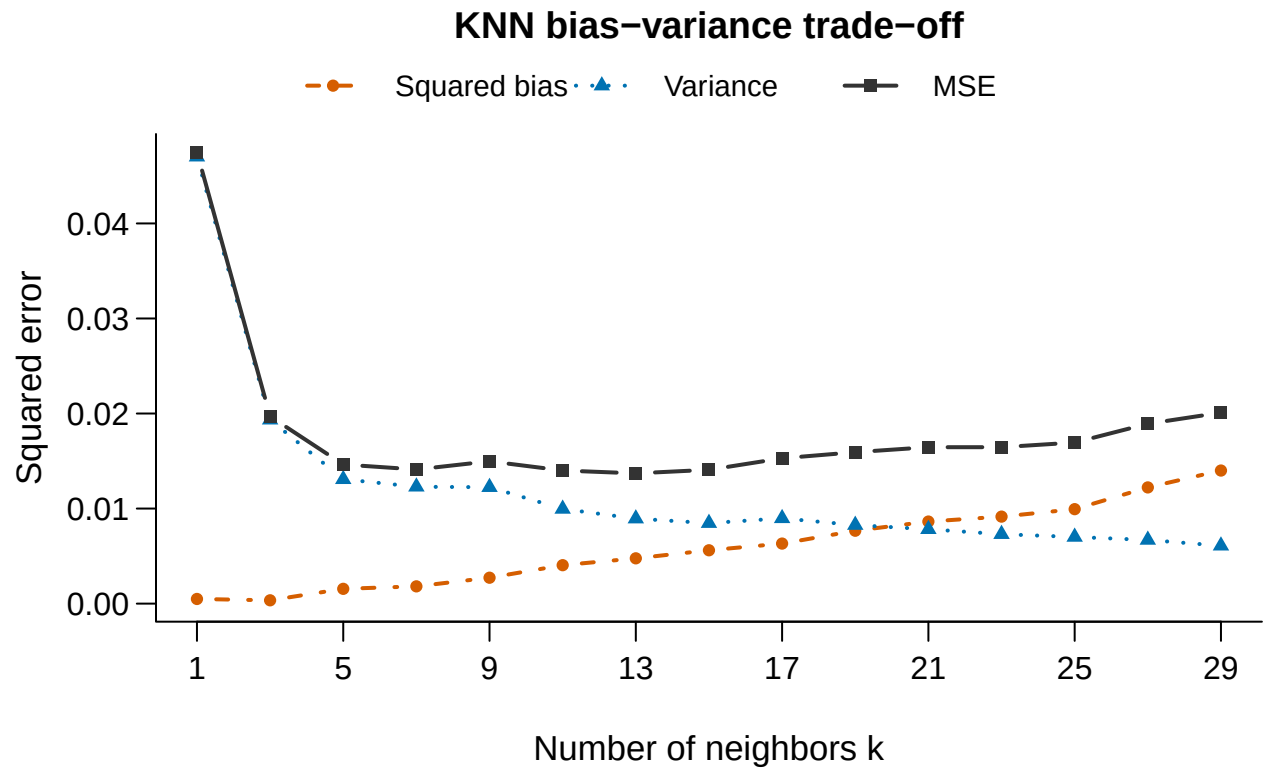
Question 5: Create a plot-formatting skill

(a)

The skill is saved in `format-homework-plots/SKILL.md`. Its description limits its use to STAT 432 homework plots and requires an explicit request.

(b)

I asked the agent to install the skill and apply it to the Question 1 plot. The original plot is retained in Question 1. The revised version below uses the same plotted values and axis ranges, with a title, thicker lines, distinct line types and point shapes, and a legend above the plotting area.



(c)

I chose these rules to improve readability while preserving the original analysis. I prefer the revised plot because it is easier to read and makes the differences between squared bias, variance, and MSE clearer. The formatting changes did not alter any data, calculations, or results, so I decided to keep the skill.

The final SKILL.md content is:

```
---
name: format-homework-plots
description: Use only when the user explicitly requests this skill to improve the formatting of
  ↳ STAT 432 homework plots in R or Python.
---

# STAT 432 Plot Formatting

- Preserve the original data, calculations, and statistical meaning.
- Make important trends and differences visually clear without exaggerating them.
- Use sufficient margins so titles, axis labels, tick labels, and legends are not clipped.
- Use a short and informative title.
- Use clear axis labels and readable tick labels.
- Use high-contrast, distinguishable colors for multiple curves, and avoid colors that are too
  ↳ similar.
- Use different point shapes or line types when multiple curves appear in the same plot.
- Use thicker lines and appropriately sized points so important details are easy to see.
- Use readable font sizes for titles, axes, and legends.
- Keep the figure size balanced and avoid unnecessary visual clutter.
- Apply these rules to existing R or Python plotting code without changing the underlying
  ↳ analysis.
```